

**MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION**

**FEDERAL STATE BUDGETARY EDUCATIONAL
INSTITUTION OF HIGHER
EDUCATION**

"UFA UNIVERSITY OF SCIENCE AND TECHNOLOGY"

Institute of Informatics, Mathematics and Robotics

Field of Study:
01.03.02 Applied Informatics

Department of Mathematical and computer modeling

Educational Program Profile:
Information and Computing Technologies

Internet of Things

Course Work

Development of a Smart Greenhouse

Supervisor:

Associate Professor, Ph.D.

TKACHEV Vladislav Igorevich

Completed by:

3th Year Student

Full-Time Education

Group PMI-PADAN 302B

BALGHOU Abdellah

UFA - 2026

**MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION
FEDERAL STATE BUDGETARY EDUCATIONAL
INSTITUTION OF HIGHER EDUCATION
"UFA UNIVERSITY OF SCIENCE AND TECHNOLOGY"**
Institute of Informatics, Mathematics and Robotics
Field of Study:
01.03.02 Applied Informatics
Department of Mathematical and computer modeling

REVIEW
of the Coursework
of student

BALGHOU Abdellah

Level of Professional Education: Higher Education – Bachelor's Degree.
Field of Study (Specialty): Applied Informatics 01.03.02
Educational Program Profile: Information and Computing Technologies.
Coursework Topic: Smart Greenhouse with Local Automation on IoT Technologies
Compliance of the aim and objectives with the coursework topic:
Complies.

Strengths of the coursework:

The coursework successfully demonstrates the development of an ESP32-based Smart Greenhouse system with automatic soil moisture monitoring, MQTT communication, and reliable control logic.

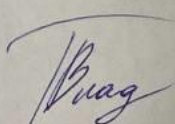
Weaknesses of the coursework:

The prototype uses LEDs instead of real actuators due to hardware limitations, and some planned features, such as weather forecast integration and a web-based monitoring interface, were not fully implemented.

The coursework complies with the requirements of the higher education educational program for professional training in this field of study and deserves the grade 5

Academic Supervisor:

Tkachev V.I.
Ass-P-PhD



"27" 06 2026

Table of content

Introduction	3
1. Theoretical Foundations of the Internet of Things	5
1.1 Internet of Things Concept	5
1.2 IoT Architecture and Layers	5
1.3 Communication Protocols	6
1.4 Wireless Technologies	7
2. Analysis of IoT Technologies and Smart Agriculture ...	11
2.1 IoT in Agriculture	13
2.2 Existing IoT Platforms	13
2.3 Comparative Analysis of Hardware Components	14
2.4 Sensor Analysis	15
2.5 IoT Communication Protocols.....	16
2.6 IoT Platforms and Cloud Services....	17
2.7 Comparative Analysis of IoT Solutions...	19
3. Development of the Smart Greenhouse System	20
3.1 Problem Statement	20
3.2 System Architecture	21
3.3 Hardware Selection	21
3.4 Software Architecture	23
3.5 Control Algorithms	27
3.6 Weather Forecast Integration	28
Conclusion	29
References	30
Appendices	33
Review	39

Relevance of the Topic

In recent years, the Internet of Things (IoT) has become one of the most rapidly developing areas in modern information technology. IoT enables physical devices, sensors, and actuators to be connected to the Internet, allowing real-time data collection, processing, and control.

One of the most important application areas of IoT is agriculture, where it is used to improve productivity, reduce resource consumption, and automate routine processes. In particular, greenhouse farming requires continuous monitoring of environmental parameters such as soil moisture, temperature, and humidity, which traditionally demands significant human effort.

The development of IoT-based smart greenhouse systems allows automation of irrigation processes, optimization of water usage, and remote monitoring of environmental conditions. This increases efficiency, reduces costs, and improves plant growth conditions.

Therefore, the development of a smart greenhouse system based on IoT technologies is a relevant and practical task.

Aim of the Work

The aim of this coursework is to develop a prototype of a Smart Greenhouse system based on IoT technologies, which performs automatic soil moisture monitoring and irrigation control using ESP32 and sensor-based data acquisition, with wireless data transmission via MQTT protocol.

Object of Research

The object of the research is Internet of Things technologies used in automated environmental monitoring and control systems.

Research Methods

The following methods were used in this coursework:

- analysis of scientific and technical literature on IoT systems;
- study of microcontroller-based embedded systems;
- comparative analysis of sensors and communication technologies;

- system design and software development using Arduino C++;
- experimental testing of the developed prototype.

Tasks of the Work

To achieve the stated aim, the following tasks were defined:

1. To study the theoretical foundations of the Internet of Things.
2. To analyze existing smart agriculture and greenhouse automation systems.
3. To select appropriate hardware components for the system.
4. To develop the system architecture of the Smart Greenhouse.
5. To implement software for ESP32-based control.
6. To design and implement soil moisture monitoring and irrigation logic.
7. To test the system and evaluate its performance.

Structure of the Work

The coursework consists of an introduction, three chapters, a conclusion, a list of references, and appendices.

- The first chapter describes the theoretical foundations of IoT.
- The second chapter analyzes existing IoT technologies and smart agriculture systems.
- The third chapter presents the practical implementation of the Smart Greenhouse prototype.
- The conclusion summarizes the results of the work.

CHAPTER 1. THEORETICAL FOUNDATIONS OF THE INTERNET OF THINGS

1.1 Concept of the Internet of Things

The Internet of Things (IoT) refers to a network of physical objects that are equipped with sensors, software, and communication capabilities, enabling them to collect and exchange data over the Internet. These objects may include embedded devices, sensors, actuators, and microcontrollers that interact with each other and with cloud-based systems.

The main idea of IoT is to extend Internet connectivity beyond traditional devices such as computers and smartphones, allowing everyday physical objects to become part of an intelligent network.

IoT systems typically operate without constant human intervention, enabling automated decision-making based on real-time data.

In agricultural applications, IoT plays a key role in monitoring environmental conditions such as soil moisture, temperature, and humidity, which are critical for plant growth.

1.2 History and Development of IoT

The concept of IoT began to develop in the late 20th century with the emergence of embedded systems and wireless communication technologies. The term “Internet of Things” was first introduced by Kevin Ashton in 1999 while working on RFID systems.

Over time, the development of low-cost microcontrollers, wireless communication protocols, and cloud computing platforms significantly accelerated the growth of IoT.

Key stages in IoT development include:

- Early automation systems (industrial control systems)
- Introduction of RFID and sensor networks
- Development of wireless communication (Wi-Fi, Bluetooth, ZigBee)
- Expansion of cloud computing services

- Modern IoT ecosystems integrating AI and big data

Today, IoT is widely used in smart homes, healthcare, industrial automation, transportation systems, and agriculture.

1.3 IoT Architecture and Layers

IoT systems are generally structured into a multi-layer architecture. The most common model consists of three layers:

1. Perception Layer

This layer includes physical devices such as sensors and actuators. It is responsible for collecting data from the environment.

Examples:

- Soil moisture sensor
- Temperature sensors
- Light sensors
- Actuators (relays, motors)

2. Network Layer

This layer is responsible for transmitting data from devices to processing systems using communication protocols.

Common technologies:

- Wi-Fi
- Bluetooth
- ZigBee
- LoRaWAN

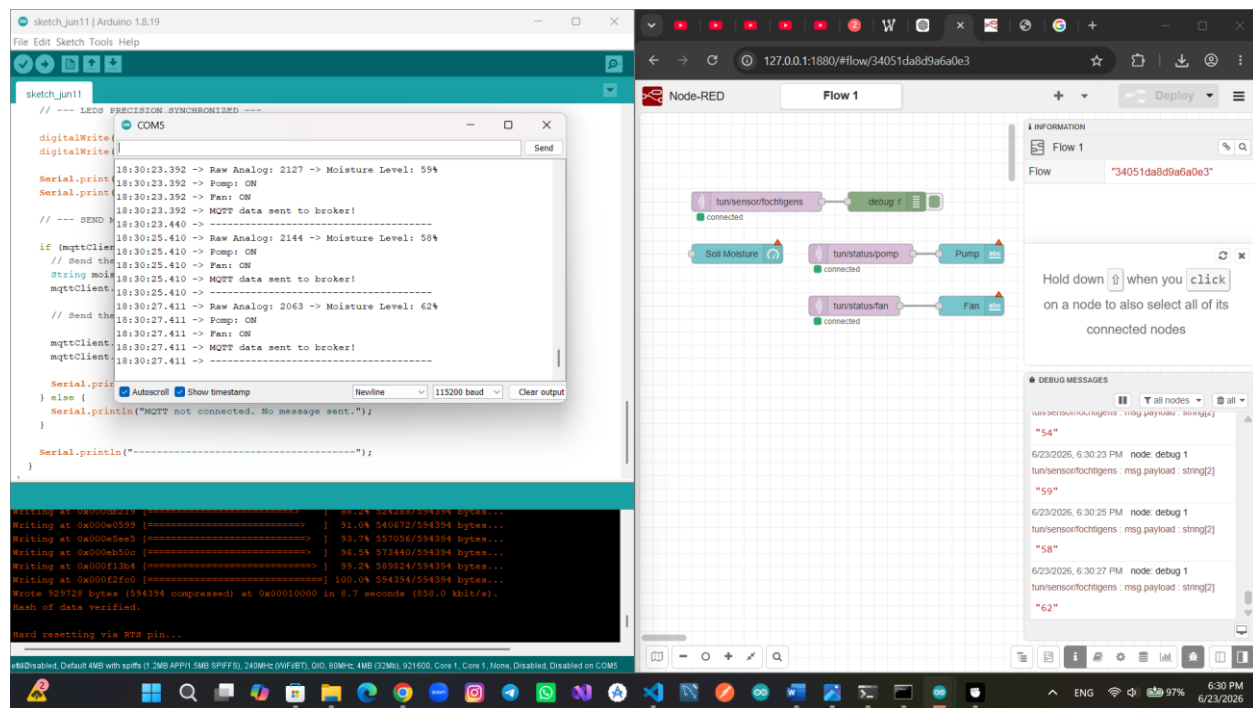
In this project, Wi-Fi is used via the ESP32 microcontroller.

3. Application Layer

This layer processes and visualizes data for users. It includes dashboards, mobile applications, and cloud platforms.

Examples:

- Web dashboards
- MQTT brokers
- Cloud services (AWS IoT, Node-RED, etc.)



1.4 IoT Communication Protocols

IoT systems rely on communication protocols to exchange data between devices and servers.

HTTP Protocol

HTTP is a widely used protocol for communication between web clients and servers. It follows a request-response model and is commonly used for REST APIs.

However, HTTP is not always efficient for IoT due to higher overhead and power consumption.

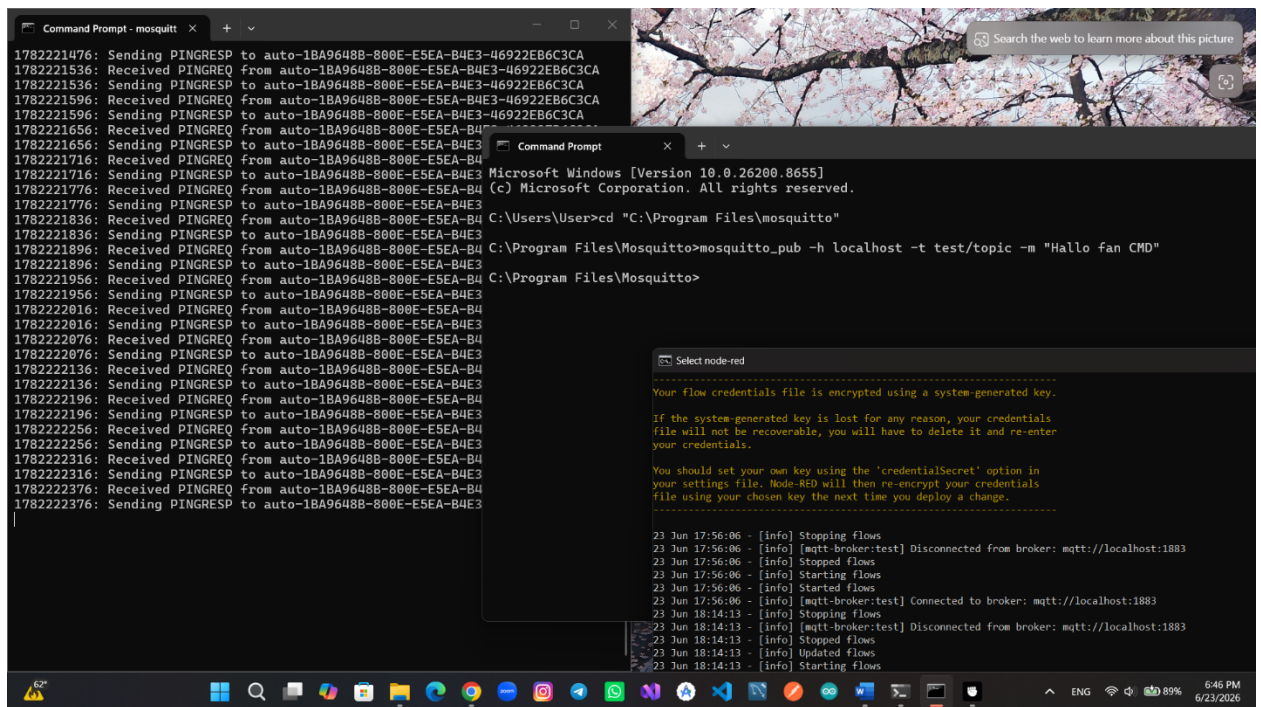
MQTT Protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe protocol designed specifically for IoT systems.

Key advantages:

- Low bandwidth usage
- Fast communication
- Suitable for low-power devices
- Reliable message delivery

In this project, MQTT is used to transmit sensor data and system status to a Mosquitto broker.



- MQTT topic messages in Mosquitto client or terminal

CoAP Protocol

CoAP (Constrained Application Protocol) is designed for constrained devices and operates similarly to HTTP but with lower overhead.

It is mainly used in low-power IoT sensor networks.

Comparison of Protocols

Protocol	Speed	Power Usage	Suitability for IoT
HTTP	Medium	High	Medium
MQTT	High	Low	Very High
CoAP	High	Very Low	High

1.5 Wireless Communication Technologies

Wireless communication is a key component of IoT systems. It enables devices to exchange data without physical connections.

Wi-Fi

Wi-Fi provides high-speed internet connectivity and is widely used in IoT systems requiring stable data transmission. ESP32 supports built-in Wi-Fi, making it suitable for smart greenhouse applications.

Bluetooth

Bluetooth is used for short-range communication with low power consumption. It is suitable for wearable and nearby device communication.

ZigBee

ZigBee is designed for low-power, low-data-rate communication in mesh networks. It is commonly used in smart home automation systems.

LoRaWAN

LoRaWAN is used for long-range communication with very low power consumption. It is suitable for agricultural and remote monitoring systems.

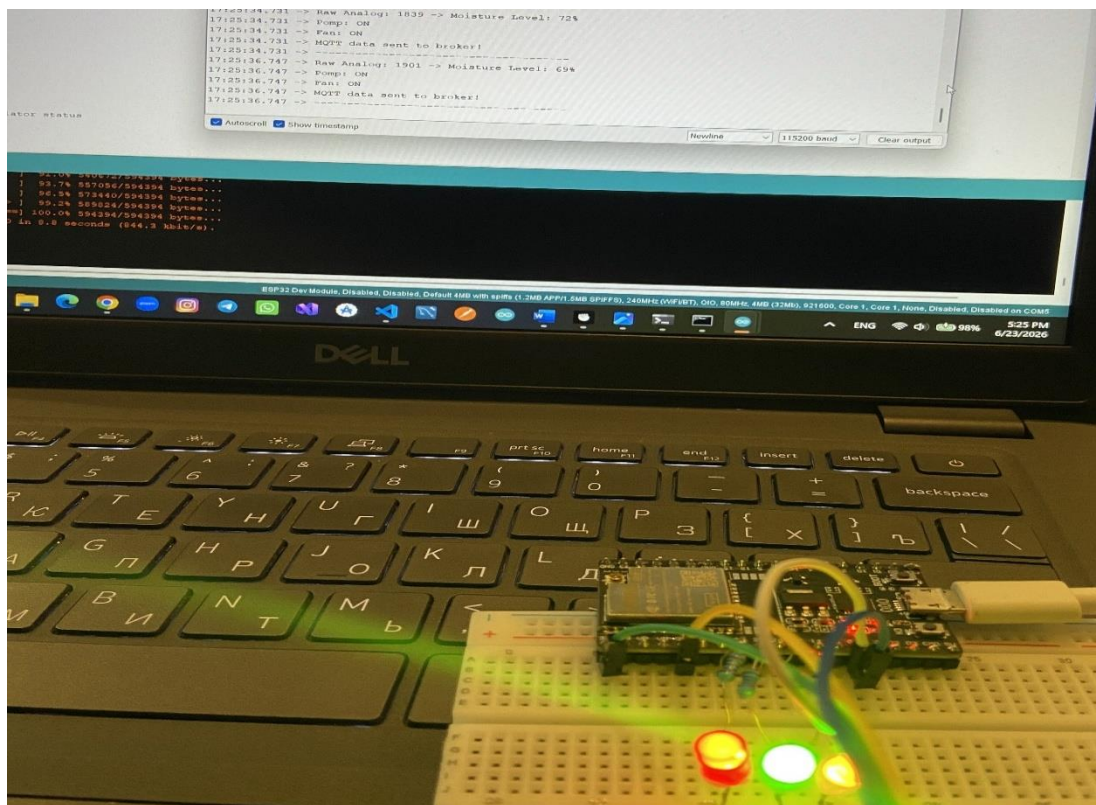
1.6 IoT in Agriculture (Smart Farming)

IoT has significant applications in agriculture, particularly in smart farming systems. These systems use sensors and automation to optimize irrigation, fertilization, and environmental control.

Key benefits include:

- Efficient water usage
- Increased crop productivity
- Reduced manual labor
- Real-time monitoring of environmental conditions

Smart greenhouses are one of the most important applications of IoT in agriculture. They allow automatic regulation of temperature, humidity, lighting, and irrigation systems.



Summary

This chapter presented the theoretical foundations of the Internet of Things, including its concept, architecture, communication protocols, and wireless technologies. Special attention was given to IoT applications in agriculture, which form the basis of the practical implementation described in the next chapters.

CHAPTER 2. ANALYSIS OF IOT MARKET AND TECHNOLOGIES

2.1 IoT in Modern Agriculture (Smart Farming)

The Internet of Things has significantly transformed modern agriculture through the introduction of smart farming systems. These systems use sensors, embedded devices, and cloud platforms to monitor and control agricultural environments in real time.

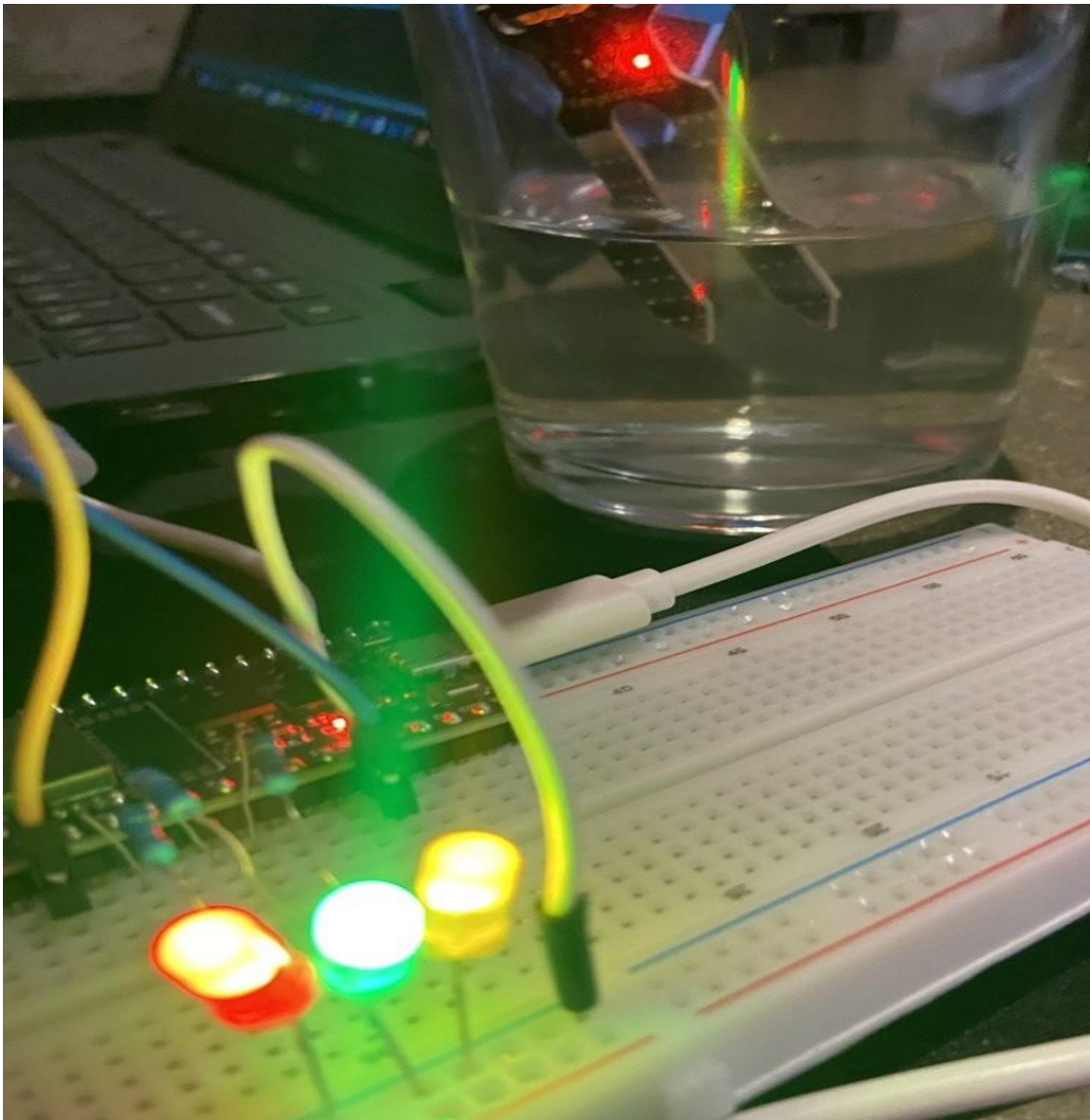
Smart agriculture focuses on optimizing resource usage such as water, energy, and fertilizers while improving crop yield and quality.

One of the most important applications is the smart greenhouse, where environmental conditions are continuously monitored and adjusted automatically.

Key functions of IoT in agriculture include:

- Real-time monitoring of soil moisture and temperature
- Automated irrigation systems
- Climate control (ventilation and heating)
- Remote monitoring through mobile or web applications
- Data-driven decision making

These technologies reduce human intervention and improve efficiency in agricultural production.



- Smart greenhouse / smart farming system overview
- IoT agriculture sensor field

2.2 Smart Greenhouse Systems

A smart greenhouse is an automated system that regulates environmental conditions required for plant growth using IoT technologies.

Unlike traditional greenhouses, smart greenhouses use sensors and controllers to automatically adjust irrigation, ventilation, and lighting based on real-time data.

Typical components include:

- Soil moisture sensors
- Temperature sensors
- Humidity sensors
- Microcontrollers (ESP32, Arduino)
- Actuators (pumps, fans, relays)
- Cloud platforms or local servers

Advantages of smart greenhouses:

- Reduced water consumption
- Improved crop growth conditions
- Remote monitoring and control
- Reduced manual labor
- Smart greenhouse interior with sensors
- Automated irrigation system

2.3 IoT Hardware Platforms

IoT systems rely on embedded hardware platforms that collect data and execute control logic.

ESP32 Microcontroller

The ESP32 is one of the most widely used IoT microcontrollers. It includes:

- Dual-core processor
- Built-in Wi-Fi and Bluetooth
- Multiple GPIO pins
- Low power consumption

It is ideal for IoT applications such as smart greenhouses due to its connectivity and performance.

- ESP32 development board

Arduino Platform

Arduino is a popular open-source electronics platform used for prototyping embedded systems.

Although simpler than ESP32, it is widely used in educational IoT projects due to its ease of use.

Raspberry Pi

Raspberry Pi is a single-board computer capable of running full operating systems.

It is more powerful than microcontrollers and is often used for:

- IoT gateways
- Data processing
- Cloud integration

Comparison of IoT Hardware Platforms

Feature	ESP32	Arduino Uno	Raspberry Pi
Processing Power	High	Low	Very High
Wi-Fi	Built-in	No	Built-in
Power Consumption	Low	Low	Medium
Cost	Low	Low	Medium/High
IoT Suitability	Excellent	Basic	Advanced

2.4 IoT Communication Technologies

IoT systems depend on communication technologies to transmit data between devices and servers.

Wi-Fi

Wi-Fi is widely used in IoT due to its high speed and availability. ESP32 supports Wi-Fi natively, making it suitable for real-time data transmission.

Bluetooth

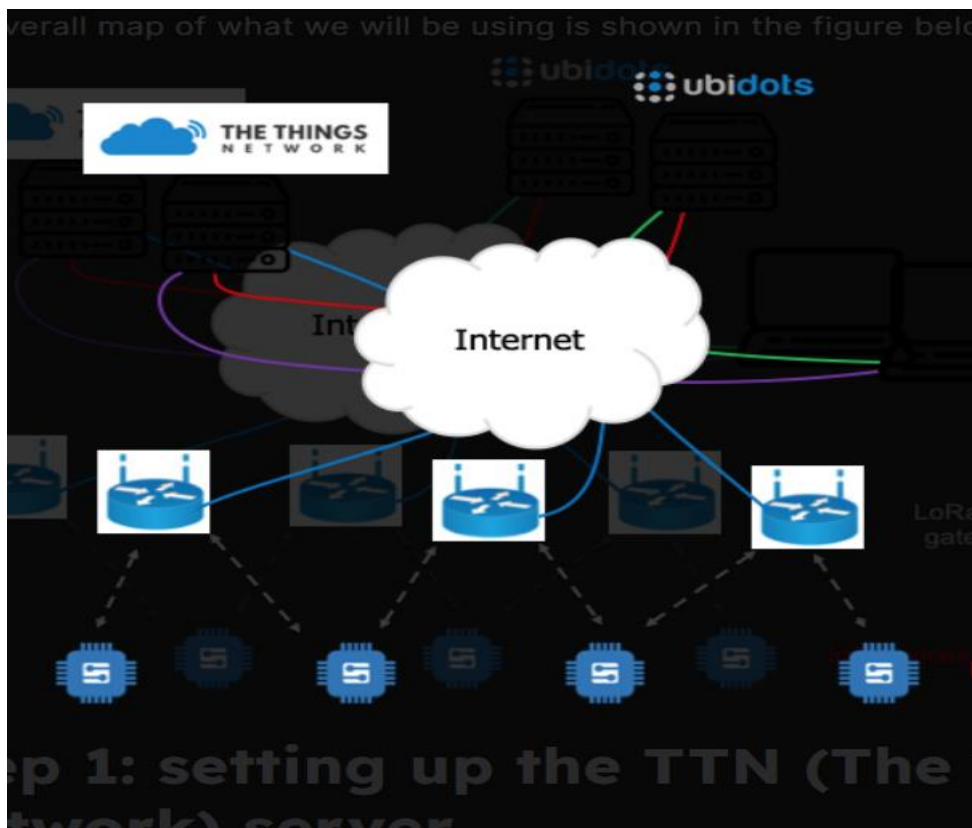
Bluetooth is used for short-range communication and low-power devices.

ZigBee

ZigBee is a low-power mesh network technology commonly used in smart home systems.

LoRaWAN

LoRaWAN is used for long-distance communication with very low power consumption, making it suitable for agriculture and remote monitoring systems.



- Wireless communication comparison diagram

2.5 IoT Communication Protocols

HTTP Protocol

HTTP is a request-response protocol commonly used in web applications. It is simple but not optimized for IoT due to higher energy consumption.

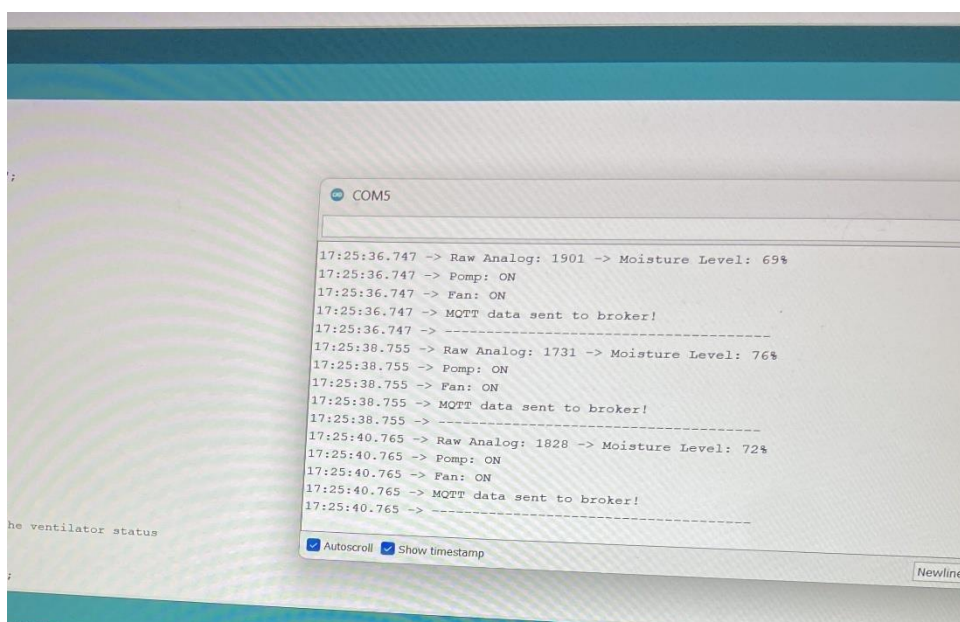
MQTT Protocol

MQTT is a lightweight messaging protocol designed for IoT systems. It uses a publish-subscribe model.

Advantages:

- Low bandwidth usage
- Fast communication
- Suitable for embedded devices
- Reliable message delivery

In this project, MQTT is used to send sensor data and system status to a Mosquitto broker.



- Mosquitto broker running in terminal
- MQTT messages received (if available)

CoAP Protocol

CoAP is a lightweight alternative to HTTP designed for constrained IoT devices.

Protocol Comparison

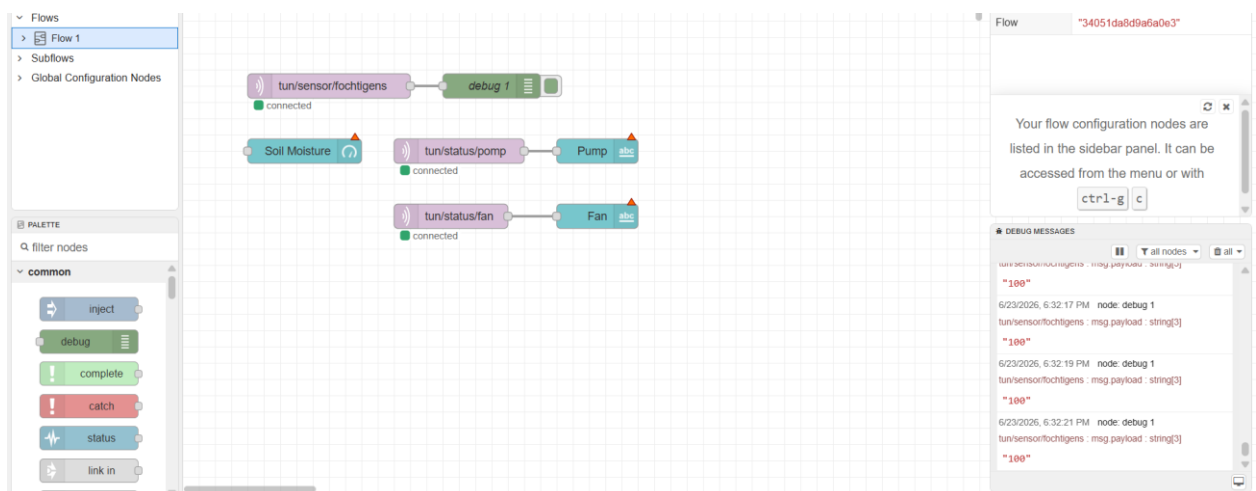
Protocol	Efficiency	Complexity	IoT Suitability
HTTP	Medium	Low	Medium
MQTT	High	Medium	Very High
CoAP	High	High	High

2.6 IoT Platforms and Cloud Services

IoT platforms provide infrastructure for data storage, visualization, and device management.

Node-RED

Node-RED is a flow-based programming tool used for IoT applications. It allows visual creation of data flows between devices and services.



AWS IoT Core

AWS IoT Core is a cloud platform for managing IoT devices and processing large-scale IoT data.

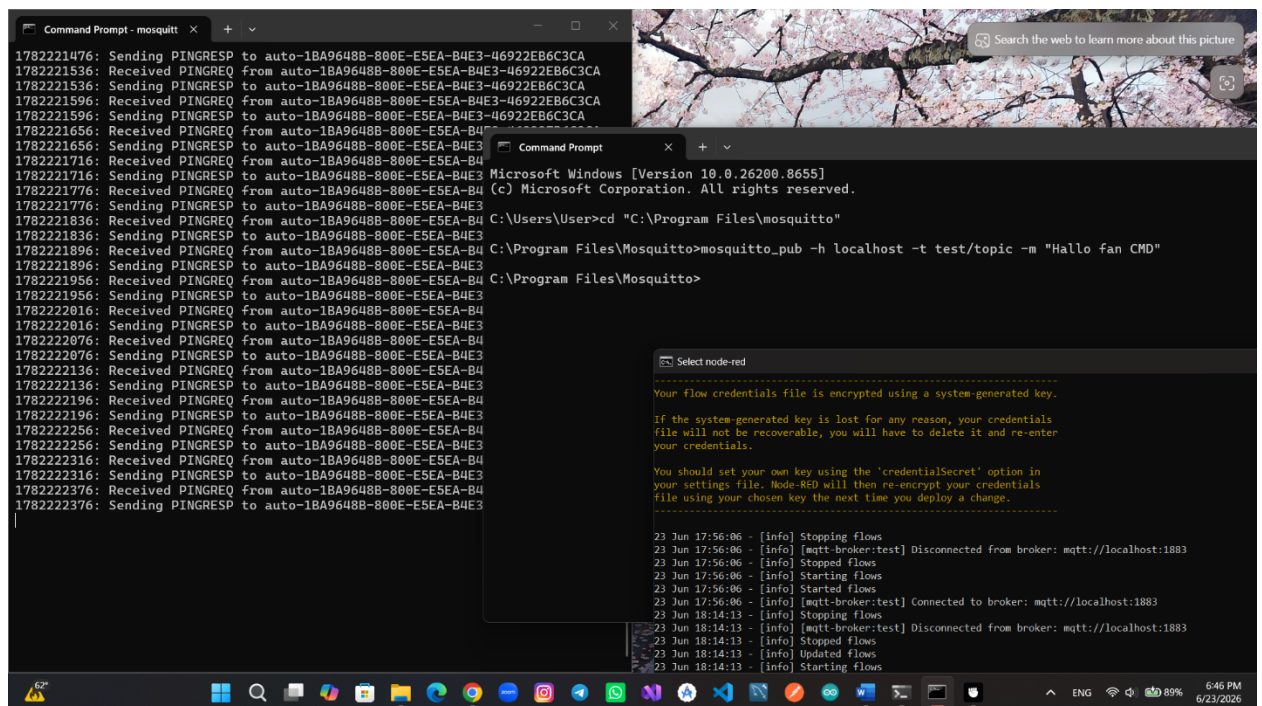
Google Cloud IoT

Google Cloud IoT provides device management and data analytics tools for IoT systems.

Mosquitto MQTT Broker

Mosquitto is a lightweight MQTT broker used for handling message communication between IoT devices.

In this project, Mosquitto is used as the central communication server for transmitting sensor data.



The screenshot displays a Windows desktop environment. On the left, a Command Prompt window titled 'Command Prompt - mosquitto' is open, showing a series of MQTT messages being sent and received between a client and a broker. The messages are timestamped and include details about the sender and receiver. On the right, a Node-RED interface is visible, showing a flow configuration. The interface includes a 'Select node-red' button and a message box indicating that the flow credentials file is encrypted using a system-generated key. Below this, there are logs showing the status of the MQTT broker connection, including 'Stopping flows', 'Disconnected from broker: mqtt://localhost:1883', 'Starting flows', and 'Connected to broker: mqtt://localhost:1883'.

- Mosquitto broker running
- MQTT client showing published topics

2.7 Comparative Analysis of IoT Solutions

Solution	Type	Advantages	Disadvantages
ESP32	Hardware	Cheap, Wi-Fi built-in	Limited processing power
Raspberry Pi	Hardware	Powerful, flexible	Higher power consumption
MQTT	Protocol	Lightweight, fast	Requires broker setup
HTTP	Protocol	Easy to use	Heavy for IoT
AWS IoT	Cloud	Scalable	Complex setup

CHAPTER 3. PRACTICAL IMPLEMENTATION OF THE SMART GREENHOUSE SYSTEM

3.1 Problem Statement

The purpose of the practical part of this coursework is to design and implement a simplified Smart Greenhouse prototype based on the ESP32 microcontroller, capable of monitoring soil moisture and automatically controlling irrigation and ventilation indicators.

In traditional agriculture, soil moisture control is often performed manually, which can lead to inefficient water usage and suboptimal plant growth conditions. To solve this problem, an automated system was developed to continuously monitor soil moisture levels and respond accordingly.

The system is designed to operate in real time, where sensor data is periodically collected, processed, and used to control output devices. Additionally, the system supports wireless communication using the MQTT protocol, enabling remote monitoring of environmental data.

Due to hardware limitations, physical actuators such as pumps and fans were replaced with LEDs, which simulate the behavior of real greenhouse devices.

The main functions of the system include:

- Measurement of soil moisture level using a capacitive sensor;**
- Automatic control of irrigation (simulated by LED);**
- Automatic control of ventilation (simulated by LED);**
- Wi-Fi connectivity using ESP32;**
- Data transmission to a Mosquitto MQTT broker;**
- Real-time monitoring through serial output.**

3.2 System Architecture

The developed system follows a layered IoT architecture consisting of sensing, processing, and communication layers.

System Components

- **ESP32 microcontroller (main controller)**
- **Capacitive soil moisture sensor (input device)**
- **LED indicators (output simulation devices)**
- **Wi-Fi network module (built-in ESP32)**
- **Mosquitto MQTT broker (communication server)**

System Operation Flow

- 1. Soil moisture sensor reads analog data from the soil.**
 - 2. ESP32 converts analog signal into percentage value.**
 - 3. The system compares the value with predefined thresholds.**
 - 4. Based on the result:**
 - **Pump LED is activated or deactivated.**
 - **Fan LED is controlled accordingly.**
 - 5. Data is transmitted to MQTT broker.**
 - 6. System repeats the cycle every 2 seconds.**
- **System block diagram (ESP32 → sensor → LEDs → MQTT broker)**

3.3 Hardware Implementation

The hardware part of the system is built around the ESP32 microcontroller, which is responsible for reading sensor data, executing control logic, and managing wireless communication.

ESP32 Microcontroller

The ESP32 is selected due to its integrated Wi-Fi module, high processing capability, and suitability for IoT applications.

Soil Moisture Sensor

A capacitive soil moisture sensor is used to measure the water content in soil. It outputs an analog signal proportional to soil moisture level.

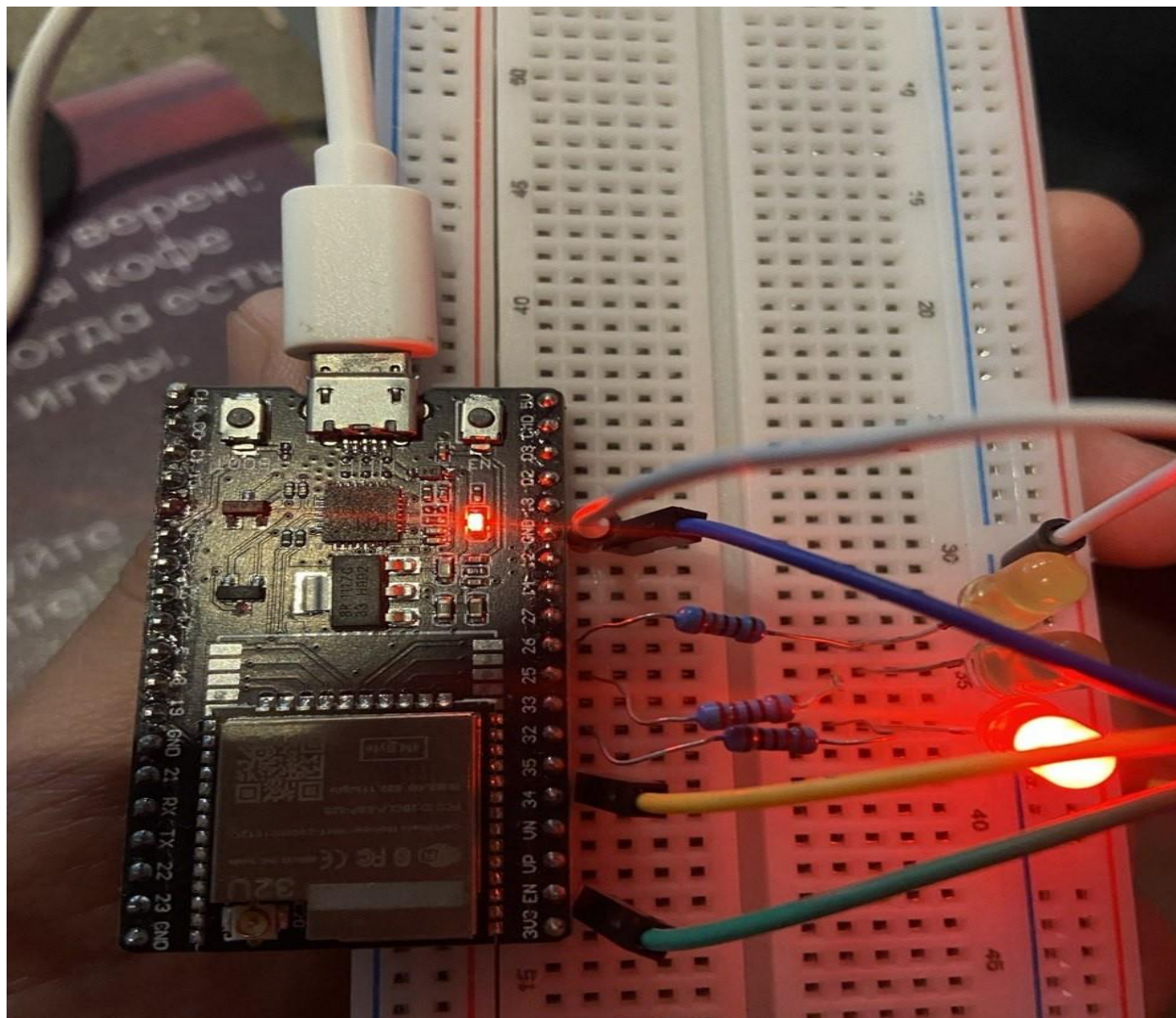
- **Connected to GPIO34 (analog input)**

- Powered by 3.3V
- Ground connected to GND

LED Indicators

Three LEDs are used in the prototype:

- Pump LED (GPIO25): simulates irrigation system
- Fan LED (GPIO26): simulates ventilation system
- Status LED (GPIO32): indicates Wi-Fi connection status



- **Photo or screenshot of hardware setup / ESP32 wiring**

Pin Configuration Table

Component	GPIO Pin	Description
Soil Moisture Sensor	GPIO34	Analog input
Pump LED	GPIO25	Irrigation indicator
Fan LED	GPIO26	Ventilation indicator
Status LED	GPIO32	Wi-Fi status

3.4 Software Implementation

The software is developed using the Arduino IDE in C++ programming language.

The system uses the following libraries:

- **WiFi.h** – for Wi-Fi connection
- **AsyncMqttClient.h** – for MQTT communication

Main Functional Logic

The system operates in a cyclic loop:

- 1. Read soil moisture value from sensor**
- 2. Convert raw analog value to percentage**
- 3. Compare with thresholds:**
 - **Below 65% → irrigation ON**
 - **Above 85% → irrigation OFF**
- 4. Control LED outputs**
- 5. Publish data via MQTT**
- 6. Repeat every 2 seconds**

Control Algorithm

sketch_jun11 | Arduino 1.8.19

File Edit Sketch Tools Help



sketch_jun11

```
mqttClient.onConnect(onMqttConnect);
mqttClient.onDisconnect(onMqttDisconnect);
mqttClient.setServer(MQTT_HOST, MQTT_PORT);

connectToWifi();
}

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - lastSensorReadTime >= s_interval) {
    lastSensorReadTime = currentMillis;

    int rawAnalog = analogRead(MOISTURE_PIN);
    int moisturePercent = map(rawAnalog, DRY_VALUE, WET_VALUE, 0, 100);
    moisturePercent = constrain(moisturePercent, 0, 100);

    Serial.print("Raw Analog: ");
    Serial.print(rawAnalog);
    Serial.print(" -> Moisture Level: ");
    Serial.print(moisturePercent);
    Serial.println("%");

    // --- PUMP LOGIC ---

    if (moisturePercent < MOISTURE_LOW_THRESHOLD) {
      pumpState = true;
    } else if (moisturePercent > MOISTURE_HIGH_THRESHOLD) {
      pumpState = false;
    }

    // --- FAN LOGIC ---

    // The fan now runs automatically when the pump delivers water
```



sketch_jun11

```
// The fan now runs automatically when the pump delivers water

if (pumpState == true) {
    fanState = true;
} else {
    fanState = false;
}

// --- LEDS PRECISION SYNCHRONIZED ---

digitalWrite(PUMP_LED, pumpState ? HIGH : LOW);
digitalWrite(FAN_LED, fanState ? HIGH : LOW);

Serial.print("Pomp: "); Serial.println(pumpState ? "ON" : "OFF");
Serial.print("Fan: "); Serial.println(fanState ? "ON" : "OFF");

// --- SEND MQTT MESSAGES ---

if (mqttClient.connected()) {
    // Send the humidity as text
    String moistureStr = String(moisturePercent);
    mqttClient.publish("tun/sensor/fochtigens", 0, false, moistureStr.c_str());

    // Send the status of the devices

    mqttClient.publish("tun/status/pomp", 0, false, pumpState ? "ON" : "OFF");
    mqttClient.publish("tun/status/fan", 0, false, fanState ? "ON" : "OFF");

    Serial.println("MQTT data sent to broker!");
} else {
    Serial.println("MQTT not connected. No message sent.");
}

Serial.println("-----");
```

- Flowchart of irrigation logic

-

Logic description:

- If moisture < 65% → Pump LED ON
- If moisture > 85% → Pump LED OFF

- **Fan LED follows pump state**

This method is known as hysteresis control, which prevents rapid switching between states when values fluctuate near the threshold.

MQTT Communication

The system connects to a Mosquitto MQTT broker using the following parameters:

- **Broker IP: 172.20.10.14**
- **Port: 1883**

Published Topics

Topic	Data
tun/sensor/fochtigens	Soil moisture percentage
tun/status/pomp	Pump state (ON/OFF)
tun/status/fan	Fan state (ON/OFF)

Code Implementation (Key Section)

The core logic of the system is implemented in the loop() function:

```

sketch_jun11 | Arduino 1.8.19
File Edit Sketch Tools Help

sketch_jun11
#include <WiFi.h>
#include <AsyncMqttClient.h>

const char* WIFI_SSID = "Abdou";
const char* WIFI_PASSWORD = "Taza$&@0610";
const char* MQTT_HOST = "172.20.10.14";
const uint16_t MQTT_PORT = 1883;

// --- PINS ---
const int MOISTURE_PIN = 34;
const int PUMP_LED = 25;
const int FAN_LED = 26;
const int STATUS_LED = 32;

// --- CALIBRATION & THRESHOLDS ---
const int DRY_VALUE = 3500;
const int WET_VALUE = 1200;
const int MOISTURE_LOW_THRESHOLD = 65;
const int MOISTURE_HIGH_THRESHOLD = 85;

AsyncMqttClient mqttClient;
unsigned long lastSensorReadTime = 0;
const unsigned long s_interval = 2000;
bool pumpState = false;
bool fanState = false; // keeps track of the ventilator status

void connectToWifi() {
  Serial.println("Connecting to Wi-Fi...");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

void connectToMqtt() {
  Serial.println("Connecting to Mosquitto MQTT Broker...");
  mqttClient.connect();
}

```

Key operations:

- analogRead(MOISTURE_PIN)
- map() function for calibration
- digitalWrite() for LED control
- mqttClient.publish() for data transmission

3.5 System Testing and Results

The system was tested under different soil moisture conditions to evaluate its performance.

Test Scenarios

Scenario	Input Condition	Expected Output	Result
Dry soil	Moisture < 65%	Pump ON, Fan ON	Success
Normal soil	65–85%	Maintain previous state	Success
Wet soil	Moisture > 85%	Pump OFF, Fan OFF	Success

Observations

- **The system responds in real time to sensor changes.**
- **LED indicators correctly simulate actuator behavior.**
- **MQTT messages are successfully transmitted to the broker.**
- **Wi-Fi reconnection works automatically.**
- **Serial Monitor output**
- **MQTT messages in broker**
- **LED state demonstration**

3.6 Fault Tolerance and System Reliability

The system includes basic fault tolerance mechanisms to ensure continuous operation.

Wi-Fi Disconnection Handling

If Wi-Fi connection is lost:

- **Status LED turns OFF**
- **ESP32 automatically attempts reconnection**
- **System continues local operation**

MQTT Failure Handling

If MQTT broker is unavailable:

- **Data is not transmitted**
- **System continues reading sensor values**
- **No interruption in control logic**

Sensor Reliability

The capacitive soil moisture sensor provides stable readings; however, calibration is required for accurate percentage conversion.

CONCLUSION

The aim of this coursework was to develop a prototype of a Smart Greenhouse system based on Internet of Things technologies, capable of monitoring soil moisture and implementing automatic control of irrigation and ventilation indicators using the ESP32 microcontroller and MQTT communication protocol.

During the development process, the theoretical foundations of the Internet of Things were studied, including its architecture, communication protocols, and areas of application. Special attention was given to IoT systems in agriculture, where automation and real-time monitoring play an important role in improving efficiency and reducing resource consumption.

In the practical part of the work, a functional prototype of a Smart Greenhouse system was designed and implemented. The system is based on the ESP32 microcontroller and a capacitive soil moisture sensor. The measured data is processed in real time, and based on predefined threshold values, the system automatically controls simulated actuators represented by LEDs.

The developed system implements the following key functions:

- continuous monitoring of soil moisture levels;
- automatic activation and deactivation of irrigation logic based on threshold values;
- simulation of ventilation control using a fan indicator;
- real-time data processing using embedded programming in Arduino C++;
- wireless data transmission using Wi-Fi;
- communication with a Mosquitto MQTT broker for telemetry exchange;
- system status indication using LED indicators.

The results obtained during testing showed that the system operates stably and responds correctly to changes in soil moisture levels. The implemented hysteresis mechanism between 65% and 85% ensures stable operation by preventing frequent switching of the control system when the sensor values fluctuate near threshold boundaries.

The use of MQTT communication allows the system to be extended in the future with cloud-based monitoring dashboards and remote control interfaces. Although advanced cloud services such as Node-RED integration and OpenWeatherMap API were not implemented in the current version, the system architecture is compatible with these technologies and can be extended in future work.

In conclusion, the developed prototype successfully demonstrates the basic principles of IoT-based smart agriculture systems. It integrates sensing, processing, and communication components into a unified system capable of autonomous operation. The project confirms the effectiveness of IoT technologies in agricultural automation and highlights their potential for future development in smart farming applications.

REFERENCES

Books and Academic Sources

1. Ashton, K. (2009). *That "Internet of Things" Thing*. RFID Journal.
2. Vermesan, O., & Friess, P. (2014). *Internet of Things: From Research and Innovation to Market Deployment*. River Publishers.
3. Tanenbaum, A. S., & Wetherall, D. (2011). *Computer Networks*. Pearson Education.
4. Hanes, D., Salgueiro, G., Grossetete, P., Barton, R., & Henry, J. (2017). *IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things*. Cisco Press.

IoT Communication and Protocols

5. MQTT Official Documentation. Available at: <https://mqtt.org/>

6. OASIS Standard – MQTT Version 3.1.1 Specification. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>
 7. Bormann, C., Castellani, A., & Shelby, Z. (2014). *CoAP: An Application Protocol for Billions of Tiny Internet Nodes*. IEEE Internet Computing.
-

ESP32 and Embedded Systems

8. Espressif Systems. *ESP32 Technical Reference Manual*. <https://docs.espressif.com/>
 9. Espressif Systems. *ESP32 Arduino Core Documentation*. <https://docs.espressif.com/projects/arduino-esp32/>
 10. Arduino Official Documentation. <https://docs.arduino.cc/>
-

IoT Platforms and Tools

11. Mosquitto MQTT Broker Documentation. <https://mosquitto.org/documentation/>
 12. Node-RED Documentation (for theoretical reference). <https://nodered.org/>
 13. IBM. *Introduction to MQTT and IoT Messaging*. <https://www.ibm.com/docs/>
-

Smart Agriculture and IoT Applications

14. Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M. J. (2017). *Big Data in Smart Farming – A Review*. Agricultural Systems, 153, 69–80.
 15. FAO (Food and Agriculture Organization). *Digital Agriculture Reports*. <https://www.fao.org/>
 16. Kumar, N., et al. (2020). *IoT in Smart Agriculture: Applications and Challenges*. Journal of Sensors and Actuators.
-

Cloud and API References

17. OpenWeatherMap API Documentation (theoretical reference). <https://openweathermap.org/api>

18. Cloud IoT Concepts Overview (AWS IoT Core).

<https://aws.amazon.com/iot-core/>

Additional Technical References

19. O'Reilly Media. *Designing Internet of Things Systems* (general IoT architecture reference).

20. IEEE Internet of Things Journal.

<https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=6488907>

APPENDICES

Appendix A – Source Code (ESP32 Smart Greenhouse System)

This appendix contains the full source code of the developed system implemented on the ESP32 microcontroller using the Arduino IDE.

```
#include <WiFi.h>
#include <AsyncMqttClient.h>

const char* WIFI_SSID = "Abdou";
const char* WIFI_PASSWORD = "*****";
const char* MQTT_HOST = "172.20.10.14";
const uint16_t MQTT_PORT = 1883;

// --- PINS ---
const int MOISTURE_PIN = 34;
const int PUMP_LED = 25;
const int FAN_LED = 26;
const int STATUS_LED = 32;

// --- CALIBRATION & THRESHOLDS ---
const int DRY_VALUE = 3500;
const int WET_VALUE = 1200;
const int MOISTURE_LOW_THRESHOLD = 65;
const int MOISTURE_HIGH_THRESHOLD = 85;

AsyncMqttClient mqttClient;
unsigned long lastSensorReadTime = 0;
const unsigned long s_interval = 2000;

bool pumpState = false;
bool fanState = false;
```

```
bool fanState = false;

void connectToWifi() {
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

void connectToMqtt() {
  mqttClient.connect();
}

void WiFiEvent(WiFiEvent_t event) {
  if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
    digitalWrite(STATUS_LED, HIGH);
    connectToMqtt();
  } else if (event == ARDUINO_EVENT_WIFI_STA_DISCONNECTED) {
    digitalWrite(STATUS_LED, LOW);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  }
}

void setup() {
  Serial.begin(115200);
```

```

pinMode(FAN_LED, OUTPUT);
pinMode(STATUS_LED, OUTPUT);

digitalWrite(PUMP_LED, LOW);
digitalWrite(FAN_LED, LOW);
digitalWrite(STATUS_LED, LOW);

WiFi.onEvent(WiFiEvent);
mqttClient.setServer(MQTT_HOST, MQTT_PORT);

connectToWifi();
}

void loop() {
  if (millis() - lastSensorReadTime >= s_interval) {
    lastSensorReadTime = millis();

    int raw = analogRead(MOISTURE_PIN);
    int moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
    moisture = constrain(moisture, 0, 100);

    if (moisture < MOISTURE_LOW_THRESHOLD) {
      pumpState = true;
    }
  }
}

```

```

    if (moisture < MOISTURE_LOW_THRESHOLD) {
      pumpState = true;
    } else if (moisture > MOISTURE_HIGH_THRESHOLD) {
      pumpState = false;
    }
  }

  fanState = pumpState;

  digitalWrite(PUMP_LED, pumpState);
  digitalWrite(FAN_LED, fanState);

  if (mqttClient.connected()) {
    mqttClient.publish("tun/sensor/fochtigens", 0, false, String(moisture).c_str());
    mqttClient.publish("tun/status/pomp", 0, false, pumpState ? "ON" : "OFF");
    mqttClient.publish("tun/status/fan", 0, false, fanState ? "ON" : "OFF");
  }
}
}

```

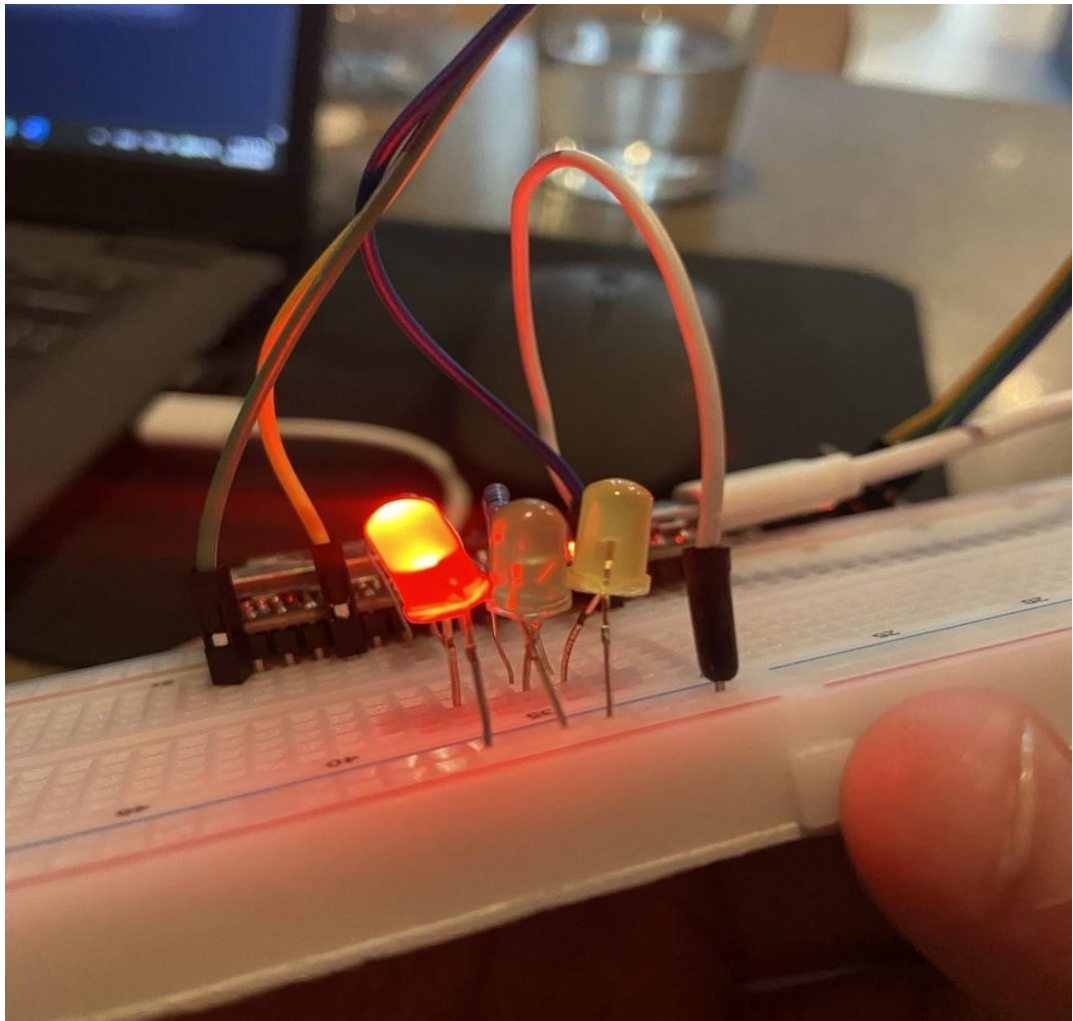
The system hardware connections are described below:

Table B.1 – Pin Mapping

Component	ESP32 Pin	Description
Soil Moisture Sensor (AO)	GPIO34	Analog input signal
Pump LED	GPIO25	Irrigation indicator
Fan LED	GPIO26	Ventilation indicator
Status LED	GPIO32	Wi-Fi connection indicator

Power Connections

- Soil moisture sensor:
 - VCC → 3.3V
 - GND → GND
- LEDs:
 - Connected through current-limiting resistors (220Ω recommended)
 - Cathode → GND
 - Anode → GPIO pins



Appendix C – System Screenshots

This appendix contains visual evidence of system operation.

Figure C.1 – ESP32 Serial Monitor Output

- Displays raw sensor readings
- Shows moisture percentage conversion
- Shows pump and fan status

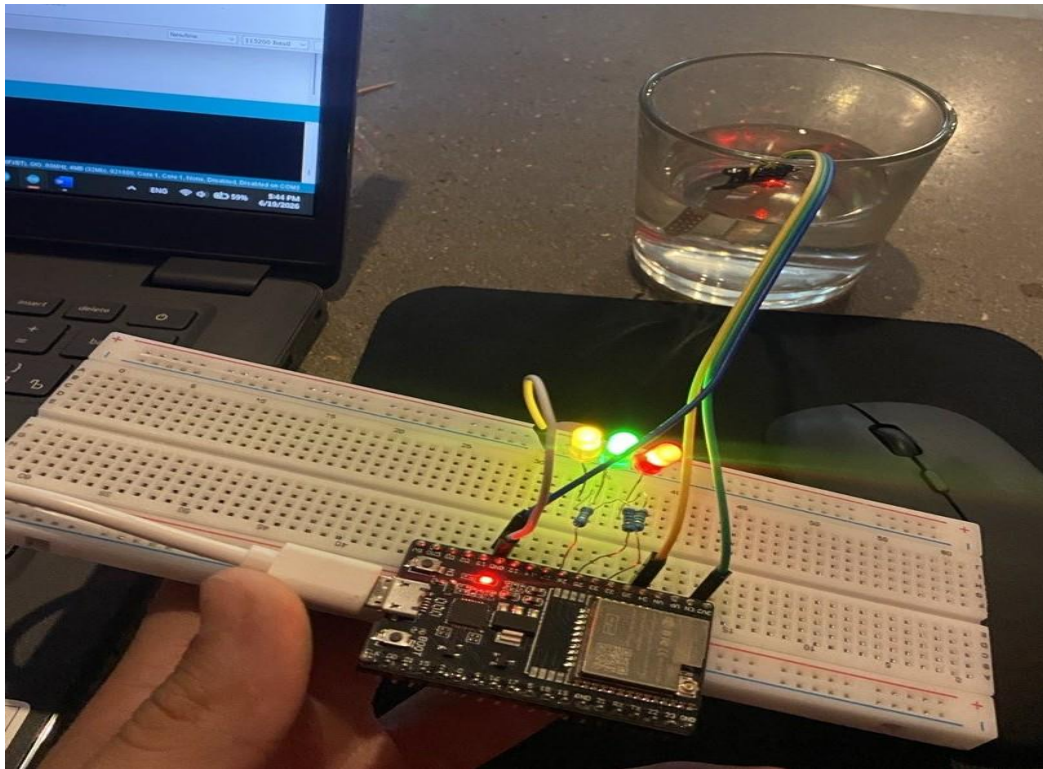
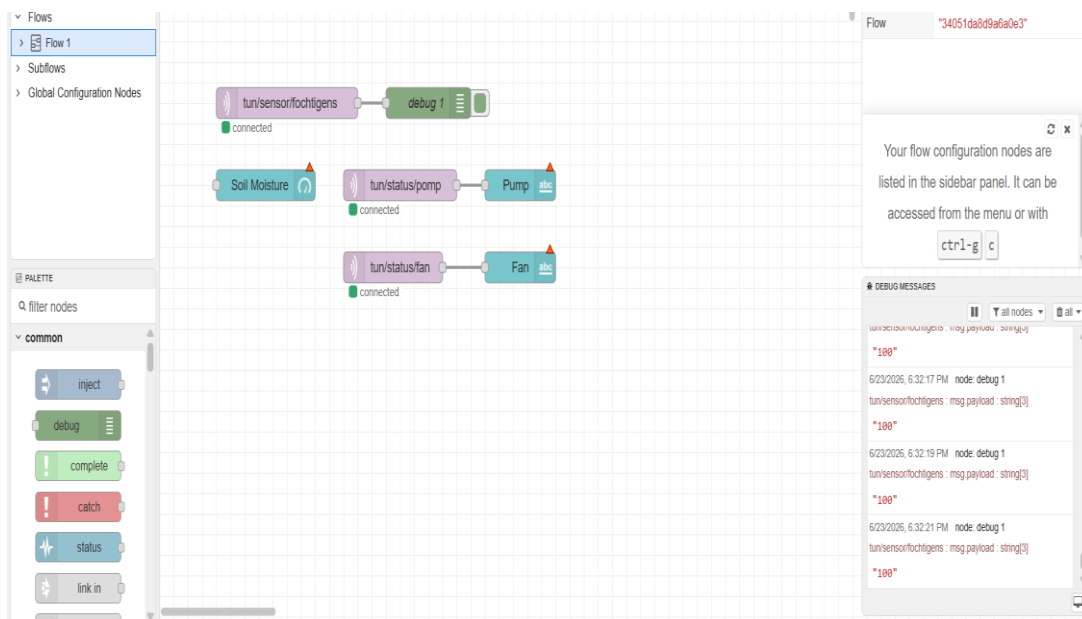


Figure C.2 – MQTT Broker Output (Mosquitto)

- Displays published topics:
 - tun/sensor/fochtigens
 - tun/status/pomp
 - tun/status/fan



Appendix D – System Operation Summary

The developed system operates according to the following logic:

1. The ESP32 reads soil moisture data every 2 seconds.
2. Raw analog values are converted into percentage values.
3. If moisture is below 65%, irrigation mode is activated.
4. If moisture is above 85%, irrigation is deactivated.
5. Fan LED follows pump state.
6. System status is shown via LED indicator.
7. Data is transmitted to MQTT broker if connection is active.

Appendix E – MQTT Topic Structure

Topic	Description
tun/sensor/fochtigens	Soil moisture value (%)
tun/status/pomp	Pump state (ON/OFF)
tun/status/fan	Fan state (ON/OFF)

